Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 07

Desarrollo de aplicaciones Web:
Implementación de Seguridad Básica.

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: Implementación de Seguridad Básica.

I. Capacidades

Implementar seguridad básica en aplicaciones web.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

VII. Procedimiento

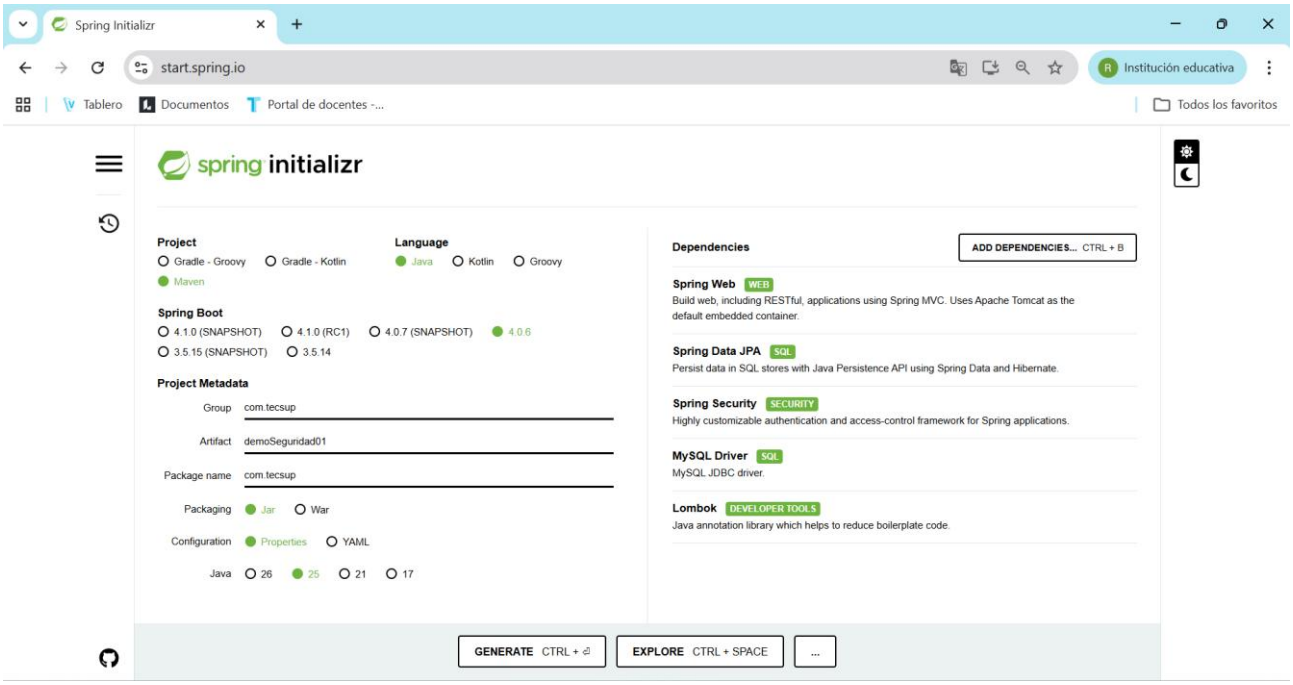
Spring Security

Base de Datos

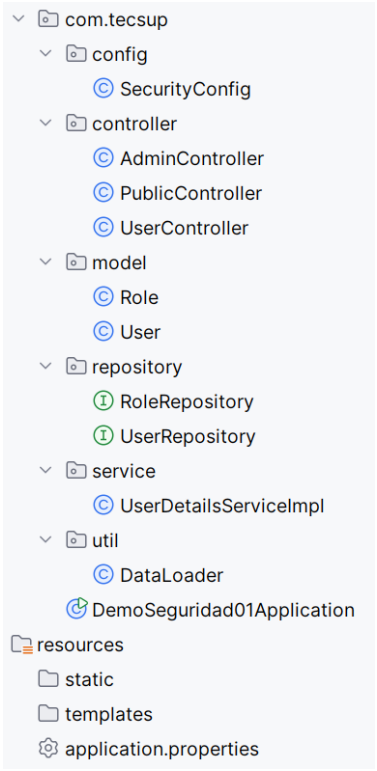
CREATE DATABASE securitydb

Backend

1. Spring Initializr



2. Crear la siguiente estructura de carpetas y archivos.



3. Clase Role

```
package com.tecsup.model;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String name; // ROLE_USER, ROLE_ADMIN

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

4. Clase User

```
package com.tecsup.model;

import jakarta.persistence.*;
import lombok.Data;

import java.util.Set;

@Entity
@Data
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
```

```
    private String username;

    @Column(nullable = false)
    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
            name = "user_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public Set<Role> getRoles() {
        return roles;
    }

    public void setRoles(Set<Role> roles) {
        this.roles = roles;
    }
}
```

5. Interface RoleRepository

```
package com.tecsup.repository;

import com.tecsup.model.Role;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;
```

```
public interface RoleRepository extends JpaRepository<Role, Long> {
    Optional<Role> findByName(String name);
}
```

6.- Interface `UserRepository`

```
package com.tecsup.repository;

import com.tecsup.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

7. Clase `UserDetailsServiceImpl`

```
package com.tecsup.service;

import com.tecsup.model.User;
import com.tecsup.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.GrantedAuthority;
import
org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundExcepti
on;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {

        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new
UsernameNotFoundException("Usuario no encontrado"));

        List<GrantedAuthority> authorities = user.getRoles().stream()
```

```
                .map(role -> new
SimpleGrantedAuthority(role.getName()))
                .collect(Collectors.toList());

        return new
org.springframework.security.core.userdetails.User(
                user.getUsername(),
                user.getPassword(),
                authorities
        );
    }
}
```

8. Clase SecurityConfig

```
package com.tecsup.config;

import com.tecsup.service.UserDetailsServiceImpl;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.authentication.configu
ration.AuthenticationConfiguration;
import
org.springframework.security.config.annotation.web.builders.HttpSecur
ity;
import
org.springframework.security.config.annotation.web.configuration.Enab
leWebSecurity;
import
org.springframework.security.config.annotation.web.configurers.Abstra
ctHttpConfigurer;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final UserDetailsServiceImpl userDetailsService;

    public SecurityConfig(UserDetailsServiceImpl userDetailsService)
{
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
```

```
        http
                .csrf(AbstractHttpConfigurer::disable)

                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/public/**").permitAll()

.requestMatchers("/admin/**").hasRole("ADMIN")

.requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                        .anyRequest().authenticated()
                )

                .userDetailsService(userDetailsService)

                .httpBasic(Customizer.withDefaults());

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {

        return config.getAuthenticationManager();
    }
}
```

9. Clase AdminController

```
package com.tecsup.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/admin")
public class AdminController {

    @GetMapping("/panel")
    public String panel() {
        return "Bienvenido ADMIN";
    }
}
```

10. Clase PublicController

```
package com.tecsup.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/public")
public class PublicController {

    @GetMapping("/hello")
    public String hello() {
        return "Endpoint público funcionando";
    }
}
```

11. Clase UserController

```
package com.tecsup.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/user")
public class UserController {

    @GetMapping("/dashboard")
    public String dashboard() {
        return "Bienvenido USER";
    }
}
```

12. Clase SecurityConfig

```
package com.tecsup.config;

import com.tecsup.service.UserDetailsServiceImpl;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.authentication.configu
ration.AuthenticationConfiguration;
import
org.springframework.security.config.annotation.web.builders.HttpSecur
ity;
import
org.springframework.security.config.annotation.web.configuration.Enab
```

```
leWebSecurity;
import
org.springframework.security.config.annotation.web.configurers.Abstra
ctHttpConfigurer;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final UserDetailsServiceImpl userDetailsService;

    public SecurityConfig(UserDetailsServiceImpl userDetailsService)
{
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {

        http
                .csrf(AbstractHttpConfigurer::disable)

                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/public/**").permitAll()

.requestMatchers("/admin/**").hasRole("ADMIN")

.requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                        .anyRequest().authenticated()
                )

                .userDetailsService(userDetailsService)

                .httpBasic(Customizer.withDefaults());

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {

        return config.getAuthenticationManager();
```

```
    }
}
```

13.- Clase DataLoader

```
package com.tecsup.util;

import com.tecsup.model.Role;
import com.tecsup.model.User;
import com.tecsup.repository.RoleRepository;
import com.tecsup.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.Set;

@Configuration
public class DataLoader {

    @Bean
    CommandLineRunner initData(UserRepository userRepo,
                               RoleRepository roleRepo,
                               PasswordEncoder encoder) {
        return args -> {

            // 🔹 Crear roles si no existen
            Role roleUser = roleRepo.findByName("ROLE_USER")
                    .orElseGet(() -> {
                        Role r = new Role();
                        r.setName("ROLE_USER");
                        return roleRepo.save(r);
                    });

            Role roleAdmin = roleRepo.findByName("ROLE_ADMIN")
                    .orElseGet(() -> {
                        Role r = new Role();
                        r.setName("ROLE_ADMIN");
                        return roleRepo.save(r);
                    });

            // 🔹 Crear usuario USER
            if (userRepo.findByUsername("user").isEmpty()) {
                User user = new User();
                user.setUsername("user");
                user.setPassword(encoder.encode("1234")); // 🔐
encriptado
                user.setRoles(Set.of(roleUser));
                userRepo.save(user);
            }

            // 🔹 Crear usuario ADMIN
```

```
            if (userRepo.findByUsername("admin").isEmpty()) {
                User admin = new User();
                admin.setUsername("admin");
                admin.setPassword(encoder.encode("123456")); // 🔐 encriptado
                admin.setRoles(Set.of(roleAdmin));
                userRepo.save(admin);
            }

            System.out.println("✔ Datos iniciales cargados correctamente");
        };
    }
}
```

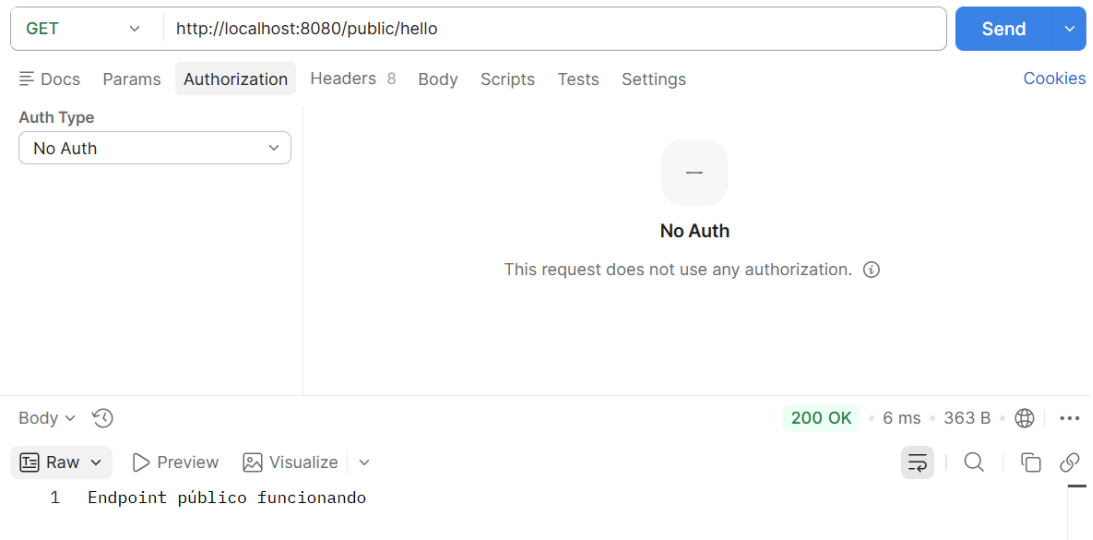
14. application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/securitydb
spring.datasource.username=root
spring.datasource.password=

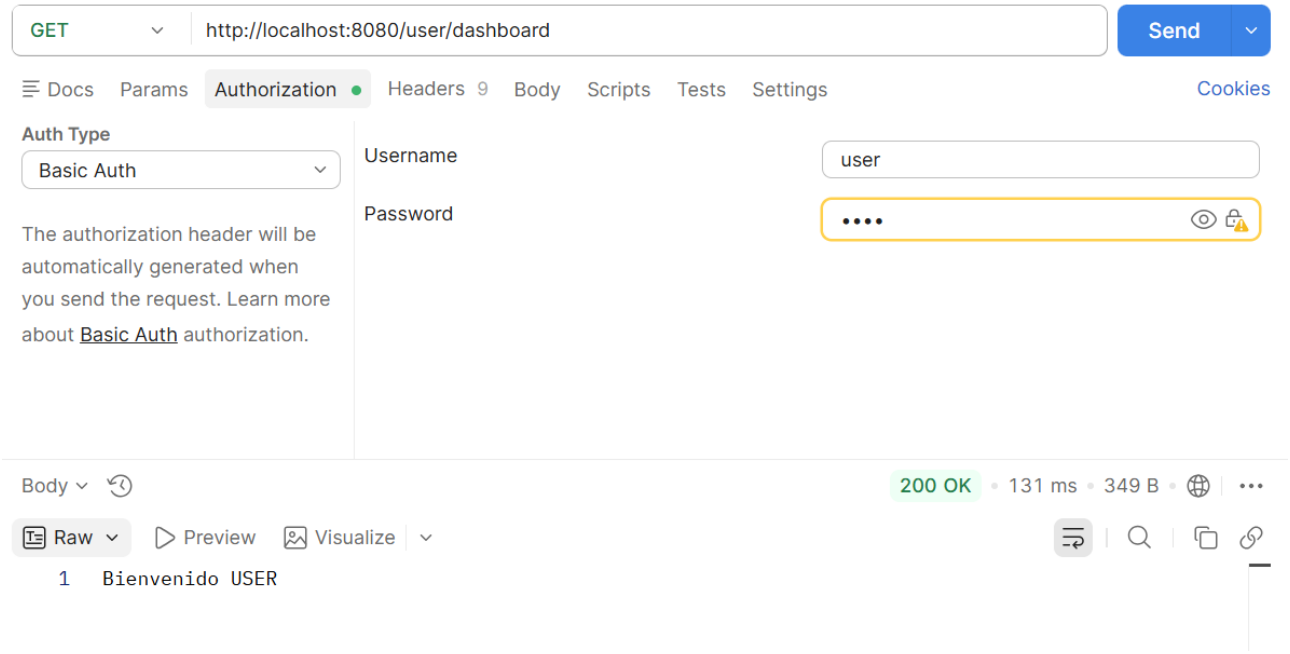
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

Pruebas en Postman

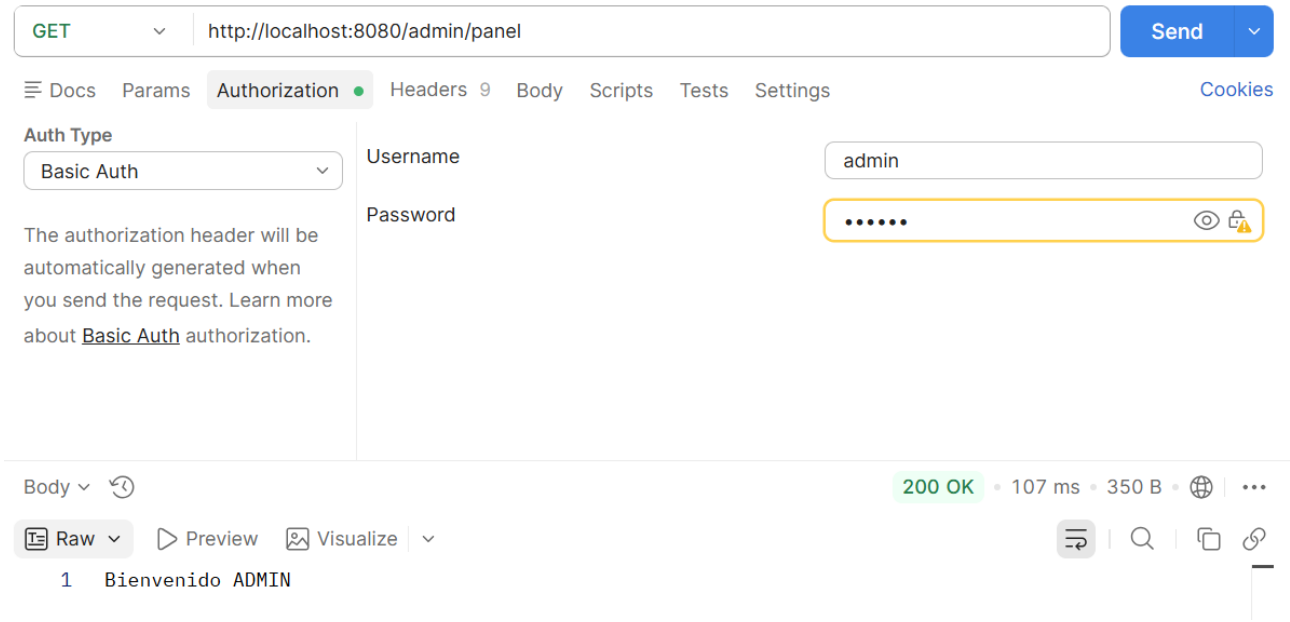
1. http://localhost:8080/public/hello



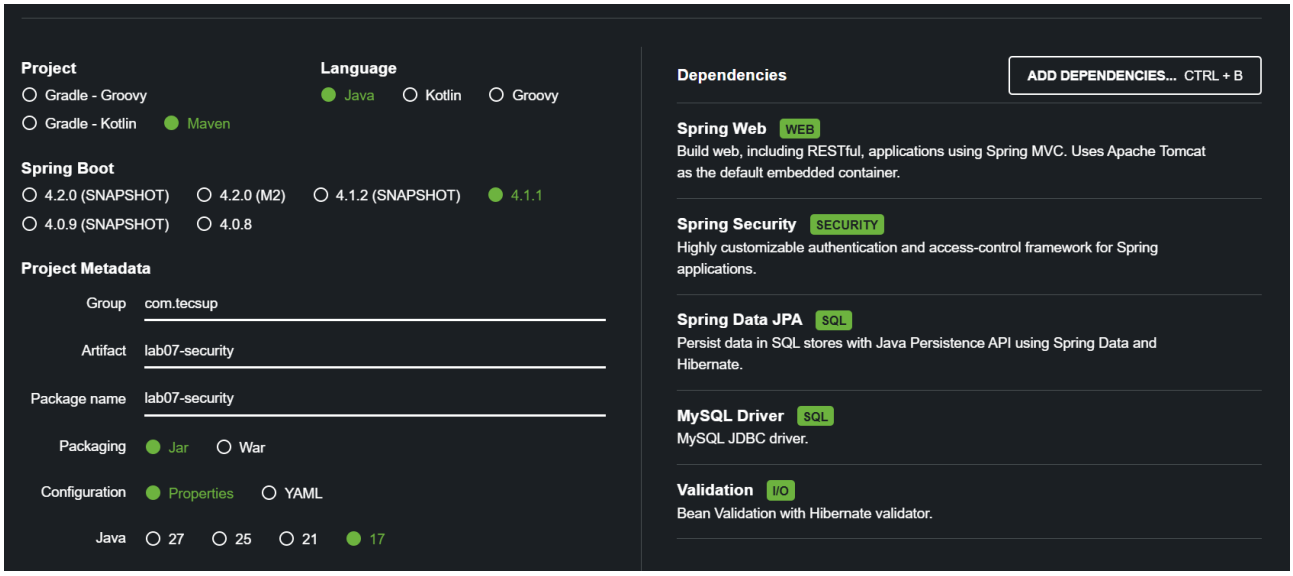
2. http://localhost:8080/user/dashboard



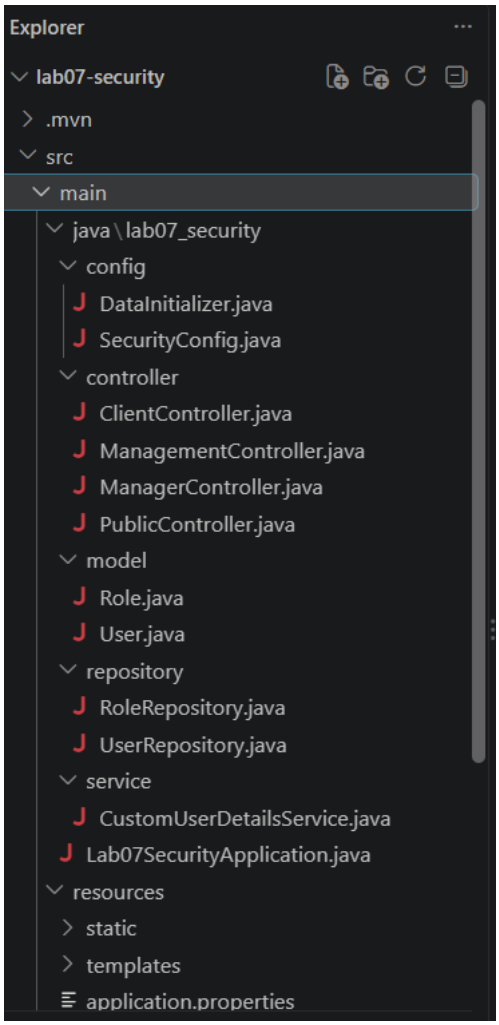
3. http://localhost:8080/admin/panel

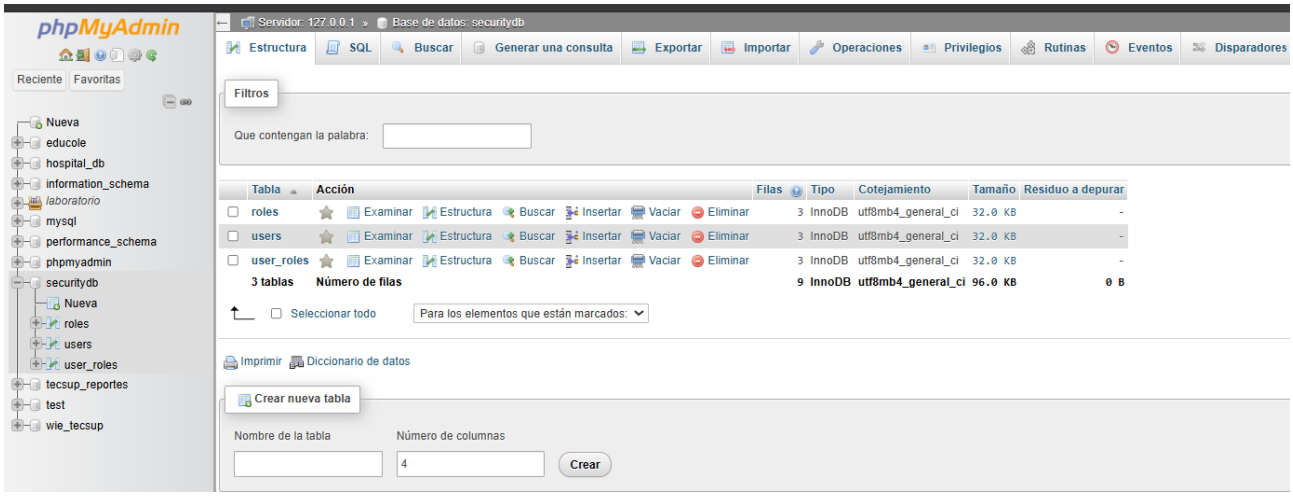


EVIDENCIAS:


Project
Gradle - Groovy
Gradle - Kotlin
Maven
Language
Java
Kotlin
Groovy
Spring Boot
4.2.0 (SNAPSHOT)
4.2.0 (M2)
4.1.2 (SNAPSHOT)
4.1.1
4.0.9 (SNAPSHOT)
4.0.8
Project Metadata
Group
com.tecsup
Artifact
lab07-security
Package name
lab07-security
Packaging
Jar
War
Configuration
Properties
YAML
Java
27
25
21
17
Dependencies
ADD DEPENDENCIES... CTRL + B
Spring Web
WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
Spring Security
SECURITY
Highly customizable authentication and access-control framework for Spring applications.
Spring Data JPA
SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
MySQL Driver
SQL
MySQL JDBC driver.
Validation
I/O
Bean Validation with Hibernate validator.


securitydb

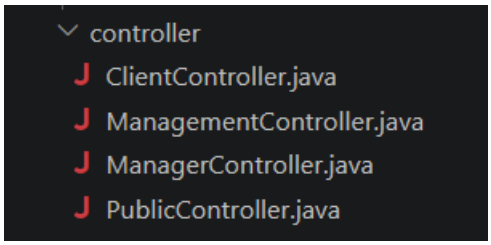

Explorer
lab07-security
.mvn
src
main
java \ lab07_security
config
DataInitializer.java
SecurityConfig.java
controller
ClientController.java
ManagementController.java
ManagerController.java
PublicController.java
model
Role.java
User.java
repository
RoleRepository.java
UserRepository.java
service
CustomUserDetailsService.java
Lab07SecurityApplication.java
resources
static
templates
application.properties



Actividad

1. Cambiar las rutas actuales.

Antes	Después
/admin/panel	/management/dashboard
/user/dashboard	/client/home
/public/hello	/api/free



CODIGO

```
package lab07_security.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class PublicController {

    @GetMapping("/api/free")
    public String free() {
        return "Endpoint público disponible";
    }
}
```

```
package lab07_security.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ClientController {

    @GetMapping("/client/home")
    public String home() {
        return "Bienvenido al área de clientes";
    }
}
```

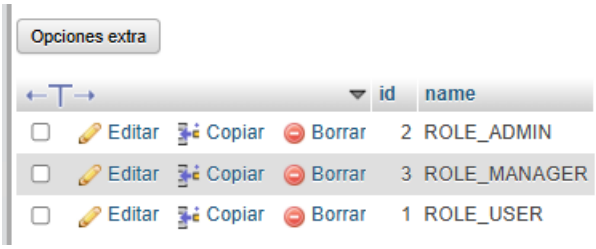
```
package lab07_security.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

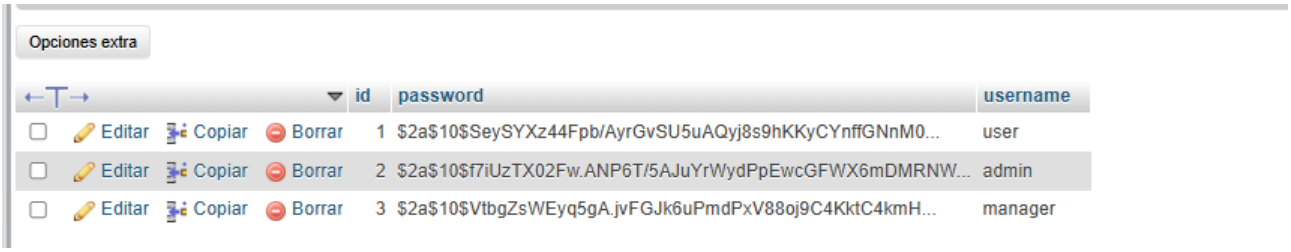
@RestController
public class ManagementController {

    @GetMapping("/management/dashboard")
    public String dashboard() {
        return "Panel de administración";
    }
}
```

2. Crear nuevo rol : ROLE_MANAGER



3. Crear un usuario : manager



4. Crear endpoint: /manager/reportes

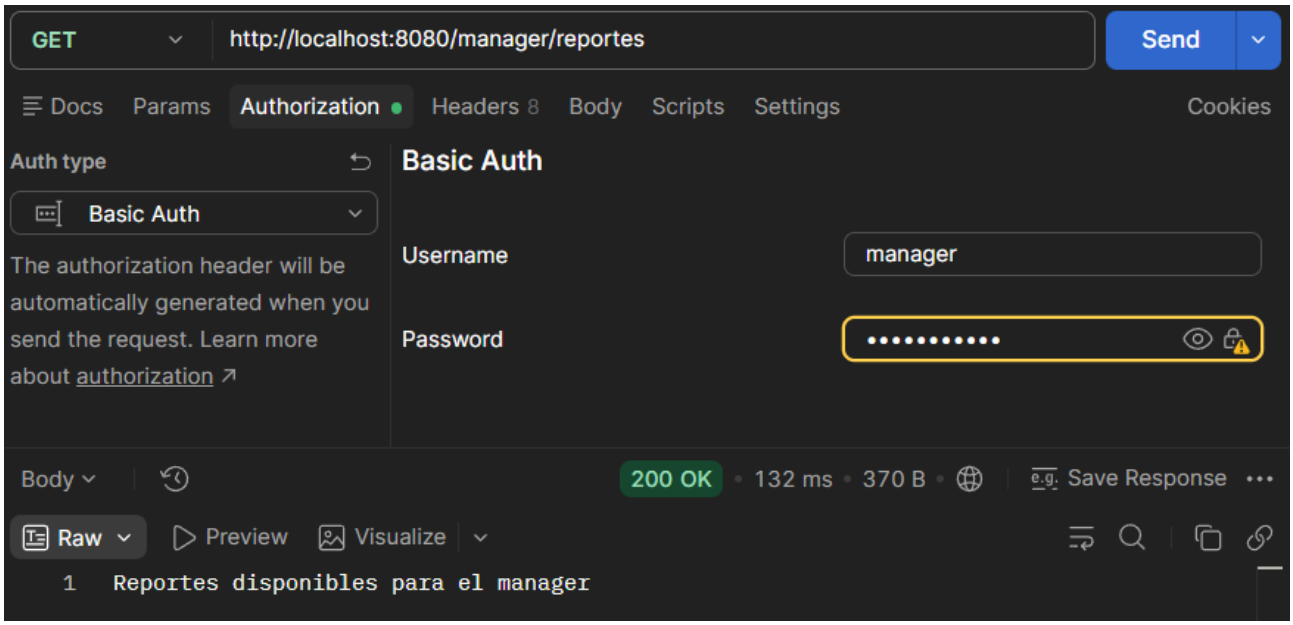
CODIGO

```
package lab07_security.controller;

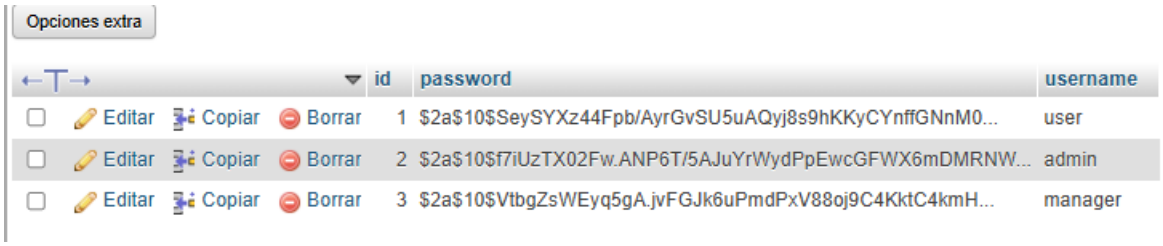
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ManagerController {

    @GetMapping("/manager/reportes")
    public String reportes() {
        return "Reportes disponibles para el manager";
    }
}
```

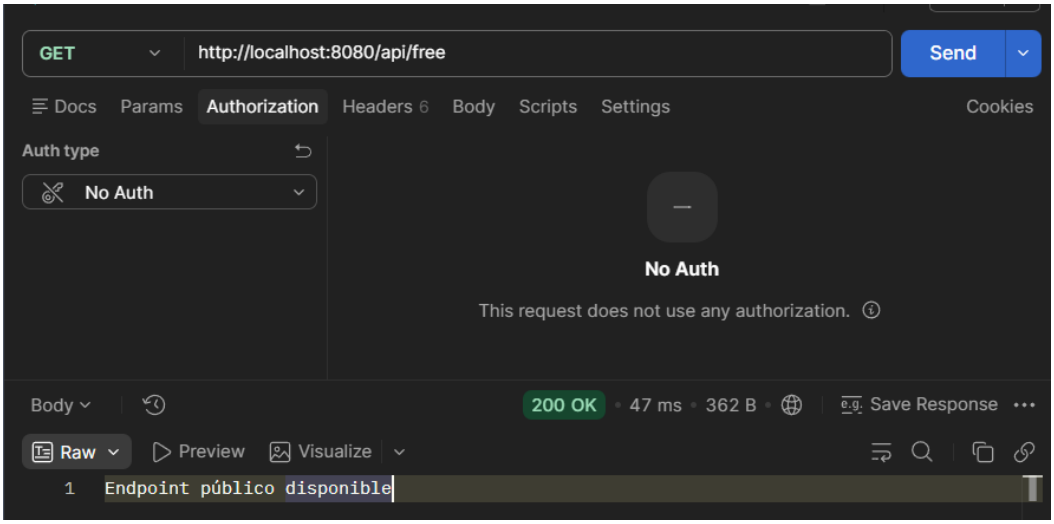


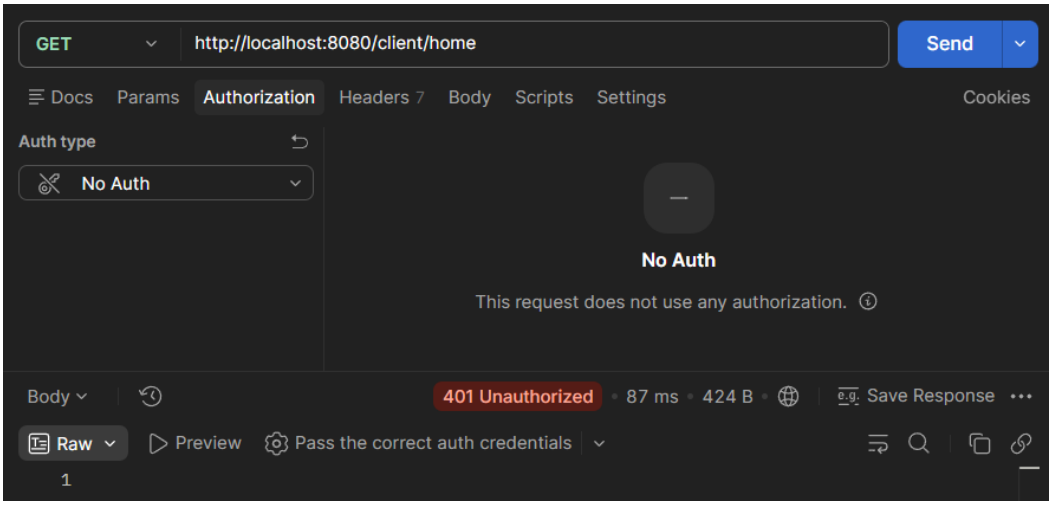
5. Cambiar contraseña a cada usuario

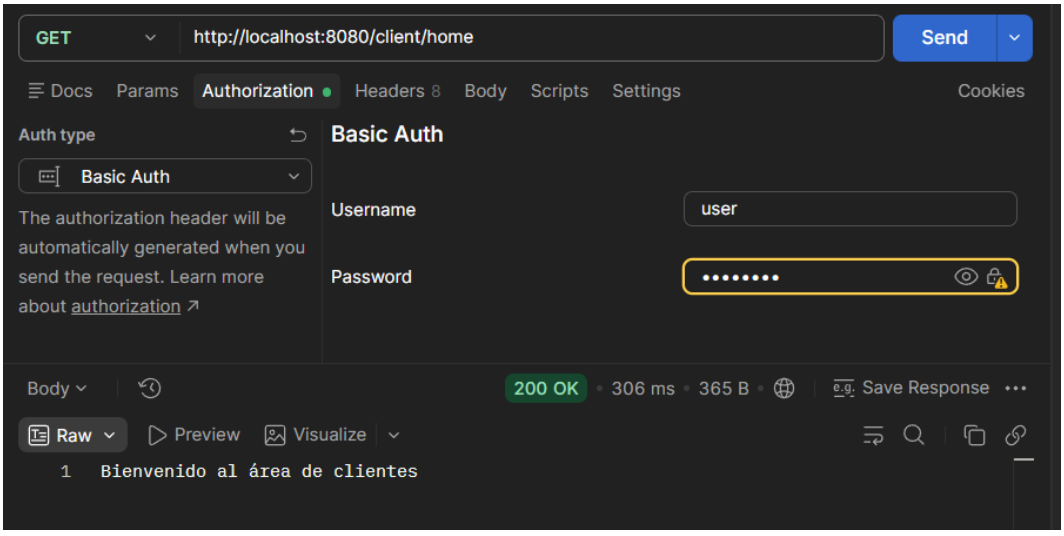


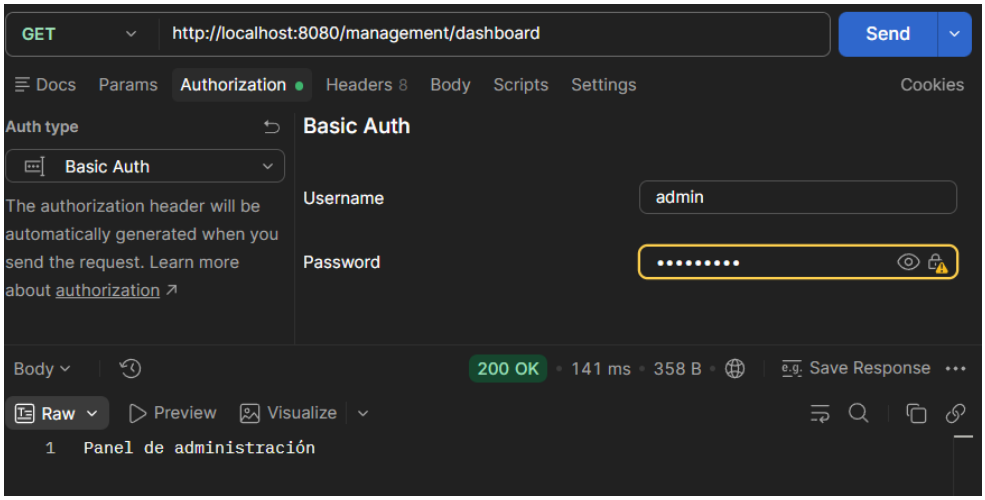
COLUMNA PASSWORD

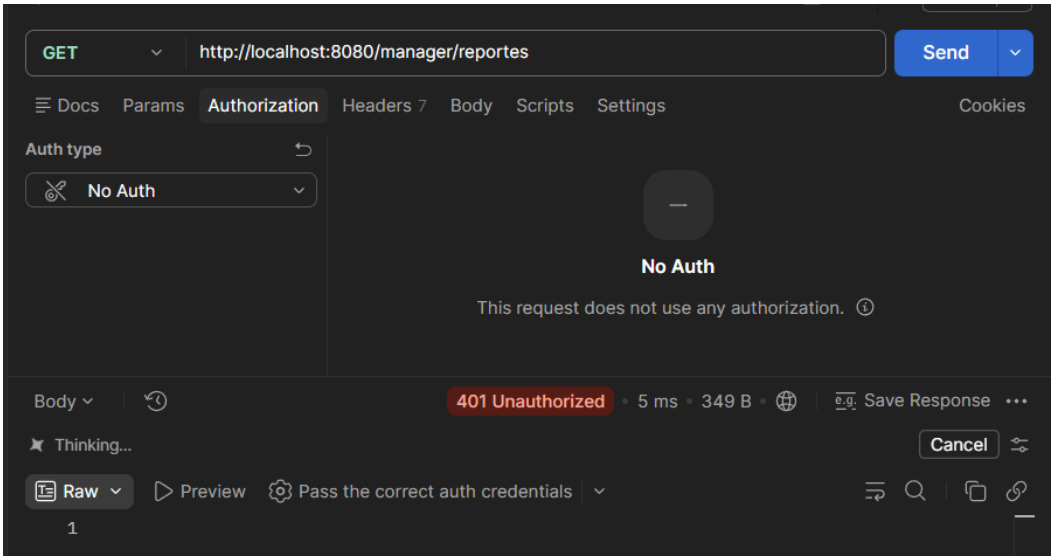
6. Realizar Pruebas en Postman

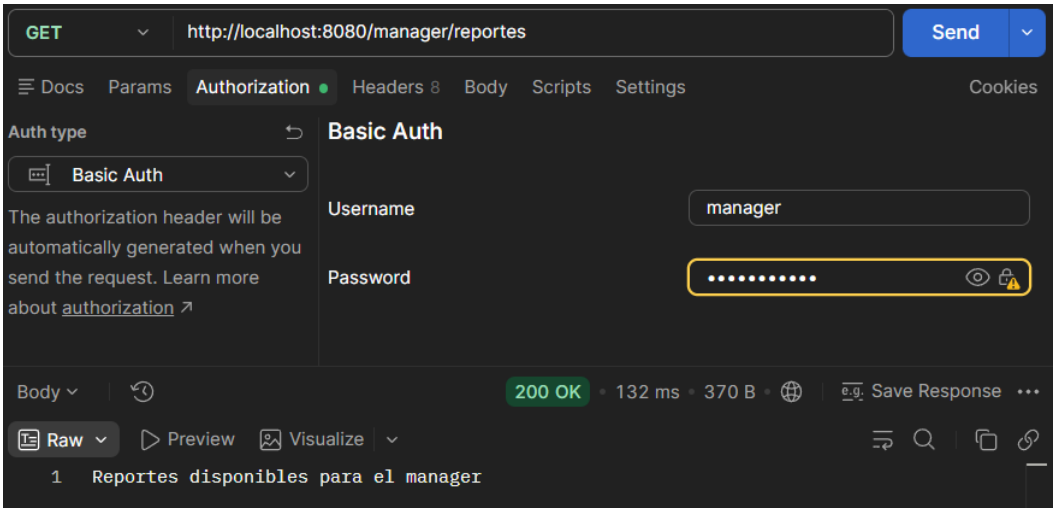


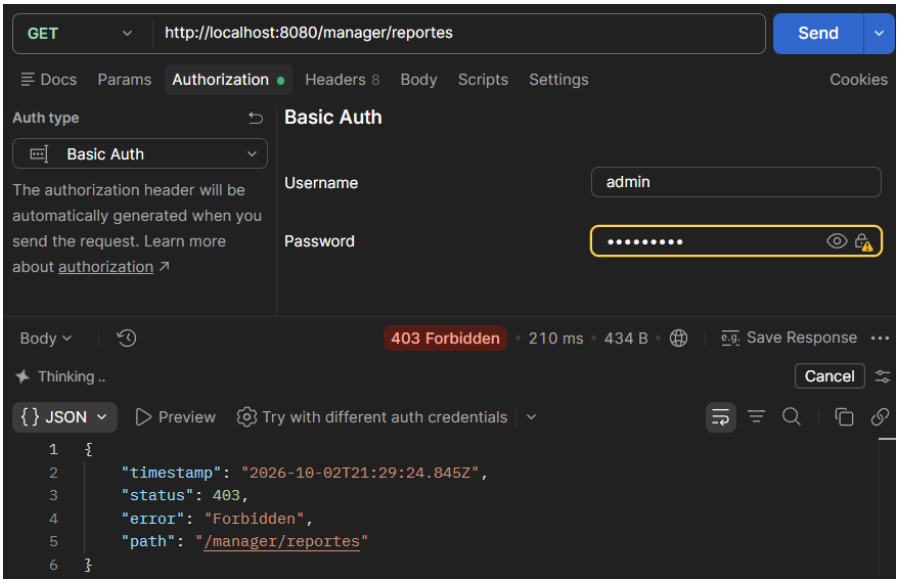


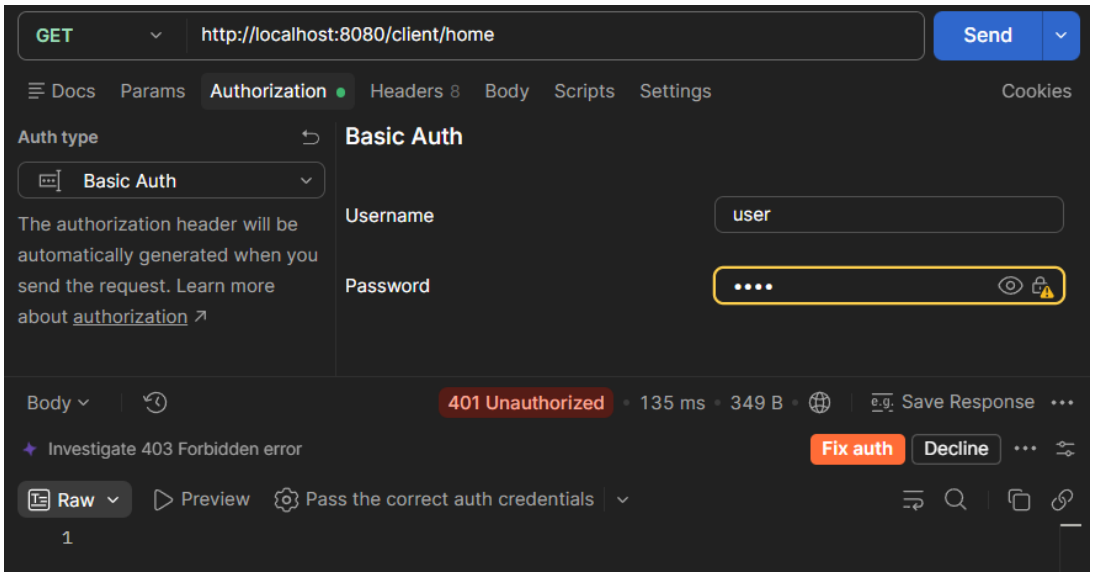



GET
http://localhost:8080/management/dashboard
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Auth type
Basic Auth
Basic Auth
The authorization header will be automatically generated when you send the request. Learn more about authorization
Username
admin
Password
Body
200 OK
141 ms
358 B
Save Response
Raw
Preview
Visualize
1 Panel de administración


GET
http://localhost:8080/manager/reportes
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Auth type
No Auth
No Auth
This request does not use any authorization.
Body
401 Unauthorized
5 ms
349 B
Save Response
Thinking...
Cancel
Raw
Preview
Pass the correct auth credentials
1


GET
http://localhost:8080/manager/reportes
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Auth type
Basic Auth
Basic Auth
The authorization header will be automatically generated when you send the request. Learn more about authorization
Username
manager
Password
Body
200 OK
132 ms
370 B
Save Response
Raw
Preview
Visualize
1 Reportes disponibles para el manager


GET
http://localhost:8080/manager/reportes
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Auth type
Basic Auth
The authorization header will be automatically generated when you send the request. Learn more about authorization
Basic Auth
Username
admin
Password
Body
403 Forbidden
210 ms
434 B
Save Response
Thinking ..
Cancel
JSON
Preview
Try with different auth credentials
{
"timestamp": "2026-10-02T21:29:24.845Z",
"status": 403,
"error": "Forbidden",
"path": "/manager/reportes"
}


GET
http://localhost:8080/client/home
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Auth type
Basic Auth
The authorization header will be automatically generated when you send the request. Learn more about authorization
Basic Auth
Username
user
Password
Body
401 Unauthorized
135 ms
349 B
Save Response
Investigate 403 Forbidden error
Fix auth
Decline
Raw
Preview
Pass the correct auth credentials
1

OBSERVACIONES: *(Las observaciones son las notas aclaratorias y problemas que se pudo presentar en el desarrollo del laboratorio)*

Mencionar 5 observaciones:

- Durante el desarrollo observé que Spring Security permite centralizar las reglas de acceso en SecurityFilterChain. Así pude definir qué rutas son públicas y cuáles requieren que el usuario se autentique, sin colocar la lógica de seguridad dentro de cada controlador.
- Al revisar la base de datos securitydb, comprobé que se crearon las tablas roles, users y user_roles. La tabla intermedia permite relacionar a los usuarios con sus roles y consultar esos permisos durante la autenticación.
- El endpoint público /api/free respondió correctamente sin credenciales. Esto me permitió comprobar que una ruta puede permanecer disponible para cualquier cliente mientras las demás siguen protegidas.
- En las pruebas de Postman observé que los códigos de respuesta dependen de la situación del usuario: 401 cuando no presenta credenciales válidas, 403 cuando está autenticado pero no tiene el rol requerido, y 200 cuando sí tiene permiso.
- La autenticación Basic facilitó las pruebas con los usuarios user, admin y manager. También observé que las contraseñas se guardan cifradas con BCrypt en la base de datos, aunque en Postman debo enviar la contraseña original para autenticarme.

CONCLUSIONES: *(Las conclusiones son una opinión sobre tu trabajo, explicar cómo resolviste las dudas o problemas presentados en el laboratorio. Además de aportar una opinión critica de lo realizado)*

Indicar 5 conclusiones:

- Con este laboratorio comprendí mejor la diferencia entre autenticación y autorización. La autenticación comprueba la identidad del usuario; la autorización verifica si su rol le permite ingresar al recurso solicitado.
- Aprendí a organizar la seguridad de una API REST mediante roles y rutas protegidas. Al definir reglas para usuario, administrador y manager, cada perfil obtuvo acceso únicamente a los endpoints que le corresponden.
- La relación entre las entidades User y Role permite asignar permisos a cada cuenta y facilita que Spring Security cargue las autoridades del usuario al iniciar una solicitud autenticada.
- Comprobé la importancia de usar BCryptPasswordEncoder para proteger las contraseñas almacenadas. De esta manera, la base de datos conserva valores cifrados en lugar de las claves originales.
- Las pruebas realizadas en Postman me permitieron verificar el comportamiento de la API en distintos escenarios. Los resultados 200, 401 y 403 me ayudaron a confirmar si el acceso era público, si faltaban credenciales o si el rol no tenía permiso suficiente.

GITHUB

- https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Web.git